■ Python Refresher

Complete Notes — Variables to Loops

#	Topic
1	Variables & Data Types
2	Operators
3	Strings & Slicing
4	String Methods
5	Lists
6	Tuples
7	Sets
8	Dictionaries
9	Loops (for / while)
10	Conditionals & Control Flow

1 · Variables

Naming Rules

- Must start with a letter or underscore (_)
- Cannot start with a digit
- Case-sensitive: myVar ≠ myvar
- Can contain letters, digits, and underscores

```python
x = 100              # single variable
x, y, z = 10, 20, 30  # multiple assignment
```

2 · Data Types

Type	Example	Description
`str`	`'hello'`	Text / string
`int`	`42`	Whole number
`float`	`3.14`	Decimal number
`bool`	`True/False`	Boolean value
`None`	`None`	Absence of value
`complex`	`2+3j`	Complex number

```python
"""  or  '''   # multi-line string
type(var)              # check type of a variable
```

3 · Type Casting

Implicit — Python converts automatically (e.g. int + float → float)

Explicit — you convert manually:

```python
int('42')    # 42
float(5)     # 5.0
str(100)     # '100'
bool(0)      # False
```

■ *Any non-zero number / non-empty string is Truthy in Python.*

4 · Operators

Arithmetic Operators

Op	Meaning	Example	Result
+	Addition	3 + 2	5
-	Subtraction	5 - 3	2
*	Multiplication	4 * 2	8
/	Division	7 / 2	3.5
//	Floor Division	7 // 2	3
%	Modulus	7 % 2	1
**	Exponent	2 ** 3	8

Comparison Operators

Operators

```
==   !=   <   <=   >   >=
```

■ *All comparison operators return a boolean (True / False).*

Assignment Operators

Operators

```
=   +=   -=   /=   *=   %=
```

Logical Operators

and	or
T T → T	T T → T
T F → F	T F → T
F T → F	F T → T
F F → F	F F → F

not — negates the boolean: `not True → False`

Membership & Identity Operators

Membership: in, not in — check if element exists in a sequence

Identity: is, is not — check if two variables point to the same object

Python

```
3 in [1, 2, 3]    # True
x is None          # True if x is NoneType
```

5 · Strings

String Slicing [start : stop : step]

- **stop** index is NOT included
- Negative indices count from the end

```python
name = 'Rohi Bindal'
name[0:2]    # 'Ro'    — chars at index 0, 1
name[:5]     # 'Rohi '
name[2:]     # 'hi Bindal'
name[:]      # full copy
name[-7:-1]  # 'Bindal'
name[::-1]   # reverse the string
```

■ *Tip: String indices: 0 1 2 3 4 … (left to right) Negative: -11 -10 … -1 (right to left)*

String Methods

Method	Description
`.upper()`	Convert all chars to uppercase
`.lower()`	Convert all chars to lowercase
`.title()`	Capitalise first letter of each word
`.capitalize()`	Capitalise only first character
`.swapcase()`	Swap upper ↔ lower case
`.find(sub)`	Return index of first occurrence (–1 if not found)
`.replace(old,new)`	Replace occurrences (1st by default)
`.strip()`	Remove leading & trailing whitespace
`.lstrip()`	Remove leading whitespace/chars
`.rstrip()`	Remove trailing whitespace/chars
`.count(sub)`	Count how many times sub appears
`.isupper()`	True if all chars are uppercase (boolean)
`.islower()`	True if all chars are lowercase (boolean)

■ *replace('Samson','Dhoni') replaces 1st occurrence; pass a 3rd arg to limit how many.*

6 · Lists

Property	Value
Ordered	✓ Yes
Mutable	✓ Yes — elements can be changed
Heterogeneous	✓ Yes — mixed types allowed
Duplicates	✓ Yes — allowed

List Operations

Python

```python
ipl_team = ['RCB', 'MI', 10, 20, True]

ipl_team[1]           # 'MI'  — access by index
ipl_team[1][0]        # 'M'   — nested index

ipl_team.append(20)           # add one element at end
ipl_team.extend(list2)        # add all elements of list2
ipl_team.insert(3, 'Dhoni')   # insert at index 3

len(ipl_team)                 # length of list

ipl_team.remove(10)           # remove by value
ipl_team.pop(8)               # remove by index (returns item)
ipl_team.clear()              # remove all elements
del ipl_team                  # delete the list entirely

ipl_team.copy()               # shallow copy
ipl_team.reverse()            # reverse in place
```

■ *Tip: copy() is a shallow copy — nested objects are still shared. Use copy.deepcopy() for a fully independent clone.*

7 · Tuples

Property	Value
Ordered	✓ Yes
Immutable	✓ Yes — cannot be changed after creation
Heterogeneous	✓ Yes
Duplicates	✓ Yes — allowed
Speed	■ Faster than lists
Memory	■ Uses less memory than lists

```python
t = (3,)            # single-element tuple — trailing comma required!
tuple.count(val)    # count occurrences
tuple.index(val)    # find index of value
del t               # delete the tuple
```

■ *Without the trailing comma, Python treats (3) as just the integer 3 in parentheses.*

8 · Sets

Property	Value
Ordered	✗ No — order not guaranteed
Mutable	✓ Yes
Duplicates	✗ No — automatically removed
Heterogeneous	✓ Yes

Python

```python
s = {1, 2, 3}

s.add(4)                    # add single element
s.update([4, 5, 6])         # add multiple elements
s.remove(2)                 # remove — raises error if not found
s.discard(2)                # remove — no error if missing

s.union(b)                  # elements in s OR b
s.intersection(b)           # elements in s AND b
s.symmetric_difference(b)   # in s OR b but NOT both
```

9 · Dictionary

Property	Value
Mutable	✓ Yes
Ordered	✓ Insertion order kept (Python 3.7+)
Heterogeneous	✓ Keys & values can be any type
Unique Keys	✓ Keys must be unique; values can repeat

Python

```python
student = {'name': 'Rohi', 'age': 25}

student.get('name')         # 'Rohi'  — safe (no KeyError)
student['age'] = 25         # access or update by key
student['height'] = 5.6     # add new key

student.keys()              # dict_keys(['name', 'age', 'height'])
student.values()            # dict_values(['Rohi', 25, 5.6])
student.items()             # dict_items([('name','Rohi'), ...])
```

■ *Use .get() over [] when a key might not exist — returns None instead of KeyError.*

Built-in Data Types — Quick Comparison

Type	Ordered	Mutable	Duplicates	Syntax

List	Yes	Yes	Yes	`[ ]`
Tuple	Yes	No	Yes	`( )`
Set	No	Yes	No	`{ }`
Dictionary	Yes*	Yes	Keys: No	`{ k:v }`
String	Yes	No	Yes	`' ' or " "`

* Dicts maintain insertion order since Python 3.7

10 · Loops

for Loop

Python

```python
# Basic range loop
for i in range(10):
    print(i)              # prints 0 to 9

# Print even numbers from a list
nums = [1, 2, 3, 4, 5, 6, 7, 8]
for i in nums:
    if i % 2 == 0:
        print(i)

# Iterate over a dictionary
for key in data.keys():            print(key)
for value in data.values():        print(value)
for key, value in data.items():    print(key, value)
```

■ *Tip: range(start, stop, step) — stop is NOT included. range(1,6) → 1,2,3,4,5 | range(0,10,2) → 0,2,4,6,8*

while Loop

Python

```python
i = 1
while i < 10:
    print(i)
    i += 1     # always update the variable to avoid infinite loop!
```

■ *A while loop without an update will run forever. Always ensure the condition eventually becomes False.*

print() — end parameter

Python

```python
print('*', end='')   # no newline — stays on same line

# Nested loop — triangle pattern
for i in range(1, 6):
    for j in range(i):
        print('*', end='')
    print()            # newline after each row
```

11 · Conditionals

```python
if condition:
    # executes when condition is True
elif another_condition:
    # executes when elif is True
else:
    # executes when none of the above match
```

12 · Loop Control Statements

```python
break      # exit the loop immediately
continue   # skip current iteration, continue loop
```

Statement	Behavior
`break`	Jumps out of the loop entirely
`continue`	Skips current iteration, continues loop
`pass`	No-op placeholder — does nothing

Bonus · Common Python Patterns

```python
# List comprehension — compact for-loop
evens = [i for i in range(10) if i % 2 == 0]

# Ternary / inline if
result = 'even' if x % 2 == 0 else 'odd'

# Enumerate — loop with index
for idx, val in enumerate(my_list):
    print(idx, val)

# Zip — loop two lists together
for a, b in zip(list1, list2):
    print(a, b)

# Check type (preferred)
isinstance(x, int)   # better than type(x) == int
```

■ *Tip: Master list comprehensions early — they replace most simple for-loops in real Python code.*